

Track-System – Backend Files Map

Last updated: 23 Sep 2026 | Covers everything up to step 3 | What each file is for and how they are wired together | **[NEW]** = added in step 3

1. Where everything sits

```
Track-System/
  Frontend/ (React + Vite - already built)
    src/pages/Login, ForgotPassword, ChangePassword
  Backend/
    TrackSystem.Api/
      TrackSystem.Api.csproj <- list of libraries
      appsettings.json       <- settings (connection string)
      Program.cs             <- the server: setup + endpoints
    Data/
      AppDbContext.cs        <- the bridge to the database
    Models/
      Employee.cs            <- [NEW] one row of dbo.Employees
    bin/ obj/ (build output - ignored by git)
  Docs/
    Backend-Files-Map.pdf    <- this file
    Backend-Glossary.pdf     <- the words to learn
  .gitignore (what git must not upload)
```

2. What each file does

File	What it is	What it does	When do I touch it
TrackSystem.Api.csproj	Project definition	Lists the .NET version and every library the project uses. Today: EntityFrameworkCore.SqlServer.	Every time we add a new library
appsettings.json	Settings file (not code)	Holds values the code needs: the connection string (server name + DB name). Because the repo is public, the future JWT key will NOT go here (see open items).	When the server name or the DB changes
Data/AppDbContext.cs	The bridge to the DB	Represents the database in C#. Every query passes through it. Holds one line (a DbSet) per table. [NEW] Now holds the Employees table.	Every time we add a table
[NEW] Models/Employee.cs	A model (a shape)	Describes one row of dbo.Employees: one property per column, with eNum marked as the primary key. It never talks to the DB by itself; EF Core fills it with data.	When a column in dbo.Employees is added, removed or renamed
Program.cs	The server	Two parts: the setup area (registers the DB, later CORS and JWT), then the endpoints (the /api/... addresses). [NEW] Now also has /api/employees.	Every new endpoint or new service
.gitignore	Git rules	Stops build files (bin, obj, node_modules) and secret files from being uploaded to GitHub.	Rarely

3. How the files are connected

```
Browser / React --(HTTP request /api/...)--> Program.cs
|
| Program.cs reads the connection string from --> appsettings.json
```

```

| Program.cs uses the classes from -----> Data/AppDbContext.cs
| AppDbContext knows each table's shape from --> Models/Employee.cs [NEW]
| all of it is possible only because -----> .csproj installed EF Core
v
AppDbContext --(EF Core turns C# into SQL)--> SQL Server, DB FinalProject
|
v
rows --> Employee objects --> only the safe fields --> JSON --> browser

```

This file	needs / uses	Why
Program.cs	appsettings.json	Asks for the value named "DefaultConnection". If the name is misspelled in either file, the connection fails.
Program.cs	Data/AppDbContext.cs	Registers it in the setup area, then receives a ready one inside each endpoint.
[NEW] Program.cs (/api/employees)	AppDbContext.Employee s	Reads the employees through the context, picks only the safe fields (no password) and sends them as JSON.
[NEW] AppDbContext.cs	Models/Employee.cs	Its Employees line says: the dbo.Employees table is made of rows shaped like Employee. It needs a using line to see the Models folder.
[NEW] Models/Employee.cs	dbo.Employees (the real table)	Property names must match the column names, or EF Core fails with "Invalid column name". If the table changes, this file must change too.
AppDbContext.cs	EF Core (from the .csproj)	It inherits from DbContext, which only exists because the library is installed.
Everything	TrackSystem.Api.csproj	If the library version does not match the .NET version, nothing builds at all.

4. The addresses that exist today (all inside Program.cs)

Address	Answers	What it proves
/api/health	{"status": "ok"}	The server itself is alive
/api/db-check	{"database": "connected"}	The server reached the FinalProject database
[NEW] /api/employees	A JSON list of employees (or [] if the table is empty). Never includes the password.	The whole chain works: DB → AppDbContext → Employee → JSON

5. Rules for this project

Rule	Why
Services are registered above var app = builder.Build();	After the server is built, nothing can be registered any more.
Server and DB names live in appsettings.json, never inside the code. Secret keys go to User Secrets (outside the repo), since the repo is public.	One place to change, the code stays clean, and nothing secret reaches GitHub.
All database traffic goes through AppDbContext	One single door to the DB, so nothing is hidden in random files.
[NEW] The model may hold the password; an endpoint never sends it	The model mirrors the table (login needs the password hash). Each endpoint chooses which fields leave the server.

Rule	Why
After a code change: save, stop (Ctrl+C), then run again	The server reads the code only at startup. This is the usual reason for a 404.
[NEW] Run dotnet commands inside Backend\TrackSystem.Api	dotnet looks for the .csproj in the current folder; from the repo root it fails.
bin/, obj/ and secret files stay out of git	They are generated automatically and would only create conflicts.

6. Open items

Item	Details
Shared server name	appsettings.json contains localhost\SQLEXPRESS. If the teammate's SQL Server has a different name, the API will not connect on his machine. To be solved later with a personal settings file.
[NEW] JWT key location	The repo is public, so the login step will store the JWT signing key in .NET User Secrets, not in appsettings.json.
[NEW] /api/employees is open	Anyone can call it today. After login exists, only the manager (or a user with permission) may see all employees. /api/db-check is a dev-only tool.
[NEW] Test data	dbo.Employees has one test row whose password is a placeholder, not a real hash. The login step will replace it with a proper hash.
Files not created yet	Models for the other 9 tables, endpoint files (to keep Program.cs short), DTO classes for the output, a Services layer for the login logic, and a test project (xUnit).